

기획서 — 조립되는 자기소개 (Assembling Bio)

컨셉

3D 공간에 흩어져 있는 큐브들을 하나씩 클릭하면, 각 큐브가 정해진 자리로 날아가 조립되면서 나에 대한 소개 정보가 하나씩 완성되는 인터랙티브 3D 자기소개 페이지.

썸 안에는 벽 역할을 하는 큰 plane 2개를 세워두고, 왼쪽 벽에는 기본 프로필(이름/직업/목표 등), 오른쪽 벽에는 skills 관련 정보가 각각 붙는 구조로 만든다. 큐브가 목표 위치(벽 근처)에 도착하면, 그 안에 담긴 텍스트/이미지/코드블럭이 큐브 표면에 직접 나타나 — 별도의 안내 문구 영역이 아니라 큐브 자체가 "완성된 조각"처럼 보이게 한다.

각 벽 그룹(왼쪽/오른쪽)의 큐브들은 처음엔 눈에 띄지 않는 무채색(그룹별로 하나의 단색)으로 흩어져 있다가, 그 벽에 속한 큐브가 전부 도착하면 그 큐브들은 사라지고 대신 그 벽에 이력서 형식의 완성된 요약 패널(제목 + 정보 목록)이 나타난다. 왼쪽/오른쪽은 서로 독립적으로 완성 여부를 판단한다.

두 벽은 서로 완전히 마주 보게 평행으로 세우지 않고, 사이드미러처럼 사용자 쪽으로 살짝 모여드는 각도로 비스듬히 세운다. 높이도 사용자 눈높이보다 훨씬 높게 잡아서, 올려다봐야 하는 압도감을 준다.

목표 / 왜 만드는가

- HTML·CSS·JS 기본기(배열/객체, 이벤트, DOM 조작) 복습
- XR 개발 방향으로 가기 위한 A-Frame(WebXR) 첫 실습
- "정답 없음" 과제이니 크게 부담 갖지 않고, 작게 시작해서 기능 하나씩 붙이기

사용 기술

- HTML / CSS / JS (ES6+)
- A-Frame (CDN으로 불러와서 사용, 별도 빌드 없음)

데이터 구조 계획

```
const introPieces = [  
  {
```

```

    id: 1,
    wall: "left",          // "left" = 프로필 벽, "right" = skills 벽
    label: "이름",
    value: "김정현",
    scatterPos: "x y z",  // 처음에 흩어져 있는 위치
    targetPos: "x y z",   // 클릭 후 조립될 최종 위치 (해당 벽 근처)
  },
  // ... 소개 항목 개수만큼 반복 (직업, 목표는 left / skills 항목들은 right)
];

// 큐브 색은 항목별이 아니라 벽 그룹별로 하나씩만 지정 (무채색)
const wallColors = { left: "#888888", right: "#555555" };

```

화면 구성 계획

- 상단: 안내 문구 ("큐브를 하나씩 클릭해서 조립해보세요")
- 중앙: A-Frame 3D 씬
 - 왼쪽에 큰 plane 벽 하나 — 기본 프로필(이름/직업/목표 등)이 붙는 자리
 - 오른쪽에 큰 plane 벽 하나 — skills 관련 정보가 붙는 자리
 - 처음엔 큐브들이 두 벽 사이 공간에 흩어져 있음
- 큐브 클릭 시: 그 큐브가 자신이 속한 벽 근처 목표 위치로 이동(애니메이션)
- 이동이 끝나면: 큐브 표면에 텍스트/이미지/코드블럭이 직접 나타남 (마치 이미지가 박힌 것처럼) — 별도 안내 영역이 아니라 큐브 자체가 정보를 담은 완성 조각처럼 보이게 함
- 상단(씬 안, 3D 팻말)에 전체 진행 상황("N / 6 조립됨")을 표시
- 한쪽 벽의 큐브가 전부 도착하면: 그 벽의 큐브들은 사라지고, 대신 그 벽에 이력서 형식의 완성 패널 (제목 + 정보 목록)이 나타남 — 왼쪽/오른쪽 독립적으로 처리

개발 단계 — 작게 시작해서 하나씩 붙이기

1. A-Frame 씬에 큐브 하나 고정 위치로 띄워보기 (기본 문법 익히기)
2. `introPieces` 배열 데이터로 큐브 여러 개를 자동 생성해서 흩어진 위치에 배치 (+ 왼쪽/오른쪽 벽 역할의 `<a-plane>` 2개 배치)
3. 큐브 클릭 이벤트가 감지되는지 확인 (우선 `console.log` 로)
4. 클릭한 큐브가 (자신이 속한 벽 근처) 목표 위치로 이동하는 애니메이션 적용
5. 이동이 끝나면 그 큐브 표면에 담긴 정보(텍스트/이미지/코드블럭)를 직접 표시 (`a-text` / `a-image` 를 큐브의 자식 엔티티로, 도착 전엔 숨김 → 도착 시 나타나게)
6. 전체적인 CSS 꾸미기 (배경, 안내 문구, 폰트 등)

7. 벽 그룹(왼쪽/오른쪽)이 각각 다 조립됐을 때, 그 벽의 큐브들을 치우고 이력서 형식의 완성 패널로 교체
8. F12 개발자 도구로 디버깅, 안 되는 부분은 `console.log` 로 값 확인
9. Netlify로 배포, URL 제출

| 위험 관리 / 대안 계획

A-Frame의 이동 애니메이션(`animation` 컴포넌트)이 예상보다 어려우면, 우선 `setAttribute` 로 위치를 "즉시 이동"시키는 방식(애니메이션 없이)으로 기능부터 완성하고, 여유가 되면 부드러운 애니메이션으로 업그레이드한다.

| 남은 디테일 작업

- ☒ 블록(소개 항목) 개수 늘리기 — 13개(프로필 7 + 스킬 6)로 확장
- ☐ 이동 애니메이션을 더 부드럽게 다듬기 — 이번 마무리에서는 보류
- ☐ 라벨(plane+text)을 큐브 표면에서 조금 더 띄우기 — 이번 마무리에서는 보류

| 진행 현황 (기록용)

- ☒ 1단계 — A-Frame CDN 연결, 큐브 하나 고정 위치로 띄우기 완료
- ☒ 2단계 — 배열 데이터로 큐브 여러 개 자동 생성 완료, 왼쪽/오른쪽 벽(plane) 배치, 벽 각도(사이드미러 각도)·높이 조정
- ☒ 3단계 — 클릭 이벤트 감지 완료, 커서(cursor) 컴포넌트로 마우스 클릭 인식 확인
- ☒ 4단계 — 클릭 시 목표 위치로 이동 + 회전(스핀) 애니메이션 적용, 벽 각도에 맞춰 착지하도록 회전 기준값 정리
- ☒ 5단계 — 이동 후 큐브 표면에 라벨(배경 판 + 텍스트)로 정보 표시, 한글 미지원 폰트 문제로 영문/ASCII로 콘텐츠 정리
- ☒ 6단계 — 전역 CSS 리셋, 상단 안내 문구, 하늘/벽 그라데이션 텍스처, `#info` 네온·float 효과까지 CSS 꾸미기 완료
- ☒ 7단계 — 벽별 완성 시 큐브를 치우고 이력서 형식 요약 패널(PROFILE/SKILLS) 표시, 진행상황 팻말(N/13 Assembled) + bounce 효과, 완성 패널·벽 tilt 애니메이션 동기화까지 완료
- ☒ 8단계 — 별도 디버깅 단계 없이 개발 전 과정에서 지켜보며 확인함 (생략)
- ☒ 9단계 — Netlify 배포 완료